

Machine Learning :- by

Andrew Ng

"Field of study that gives computers the ability to learn without being explicitly programmed"
~ Arthur Samuel (1959)

Machine learning algorithms

→ Supervised

used most in ~~real~~ real world application

→ Un Supervised

⇒ used in

- recommender systems
- reinforcement learning

Supervised Learning

⇒ When you give a machine learning algorithm an input & the right answer to that input.

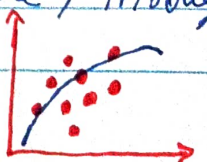
⇒ The algorithm learns how to predict the output this way

its application are :- Spam filtering, speech recognition, machine translation.

example :-

A regression model,

In a regression model you give a bunch of labeled input, ~~to~~ Output datapoint & the model tries to fit, a curve or a line, through the data points that best fits the data.



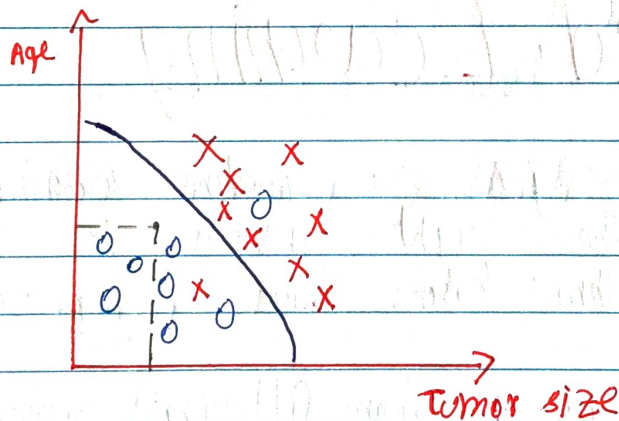
A Classification model,

in this kind of model the output class or category are very small, for example,

if you are making a breast cancer detection model, you can look at the tumor & say the tumor is not cancer tumor or it can say it is type 1 or type 2 cancer tumor, so it simply predicts where this tumor belongs, Class

- ① non-cancer tumor
- ② type 1
- ③ type 2

it can take n-number of input's like age, tumor size, tumor cell alignment ----- etc, intuitively you can say, it draws a line to distinguish between cancerous & non-cancerous tumor.



so now you can choose an age & tumor size & see if the tumor is cancerous or not.

Unsupervised learning

This type of learning takes data without labels & tries to automatically group them into clusters finding the commonality in a bunch of data & grouping that data according to that. The algorithm has to find structure in the data.

ex $\Rightarrow$ Clustering, Anomaly detection, Dimensionality reduction

Clustering $\therefore$ Group similar data points together

Anomaly detection $\therefore$ find unusual data points.

Dimensionality reduction $\therefore$ Compress data using fewer numbers.

Linear Regression model

⇒ A Linear Regression model plots a line that best fits the input & output datapoints. & through that line we can tell or estimate the output of an unknown input. it can generate " ∞ " numbers for " ∞ " number of output for " ∞ " number of inputs.

⇒ Here are some important notations in machine learning.

training set ⇒ The data that your model will be trained on

x = "input" variable or feature

y = "output" variable or target

m = number of training examples

(x, y) = single training example

$(x^{(i)}, y^{(i)})$ = i^{th} training example
(1st, second, or third - - -)

{Here i is a superscript.}

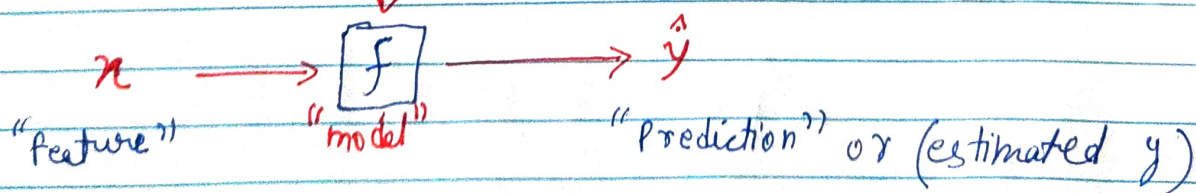
Training a machine learning model which is "linear regression"

You have, "Training set". {features & targets}

↓ goes into

"Learning algorithm"

↓ which produces



what is f ?

$$\Rightarrow f_{w,b}(x) = wx + b$$

Can also be written as,

$$f(x) = wx + b$$

This is linear Regression with one variable.
or "univariate linear regression".

Cost function

A cost function is essentially the way of computing how good a model is,

where a cost function is given by,

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

m = number of training examples.

$$\hat{y} = f_{w,b}(x^{(i)})$$

$$(\hat{y} - y) = \text{the error}$$

Here, we divide the cost function by its size taking the average because, think about it as the sample size increases the cost function will also increase & we don't want that that's why we take average & divide it by " m ".

Linear regression intuition :-

model :-

$$f_{w,b}(x) = wx + b$$

Parameters :-

w, b {weights & biases}

Cost function :-

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Goal :-

minimize $J(w,b)$
or find the value of " w & " b " that best fit's the ~~data~~ Training set.

To - minimize the cost function you need gradient descent. (it's more detailed)

Do, it from the maths for machine learning notes,[^]
if i encounter cool info i will write that down.

for learning rate " α " $\rightarrow$

if α is too small,

Gradient descent

may be slow.

if α is too large,

Gradient descent

may overshoot, never reach minimum &

fails to converge, & may even diverge.

What if you are already at a local minimum,

then.

$$w = w - \alpha \frac{\partial}{\partial w} J(w)$$

{ Notice, the equal to sign here is not for assertion it's to assign value }

So, when you are at minimum the slope becomes zero, hence the derivative is also zero, so

$$w = w - \alpha \cdot (0).$$

$$\text{So, } w = w,$$

No change.

The type of gradient descent that uses all the training examples for every step of gradient descent is called $\therefore$ Batch gradient descent

Linear regression {Multiple features} Multivariate

"Symbols & Notations"

example $\rightarrow$ Housing model,

Size in feet	Number of bedrooms	Number of floors	Age of home	Price
x_1	x_2	x_3	x_4	---
1	2	3	4	500
2	6	8	3	1000

$x_j = j^{\text{th}}$ feature (or column)

$n =$ number of features, in this example " $n=4$ "

$x^{(i)} =$ features of i^{th} training ~~feature~~ example, it is a vector for example $x^{(2)} = [2, 6, 8, 3]$

$x_{(j)}^{(i)} =$ Value of feature j in i^{th} training example,

$$Cx \ni \boxed{\begin{matrix} x_{(1)}^{(1)} \\ x_{(3)}^{(1)} = 3 \end{matrix}}$$

$j =$ Subscript denoting column

$i =$ Superscript denoting rows

The linear regression model with multiple variables will look like

$$f_{w,b}(x) = w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b$$

Here, you can say that ----- umm i forgot took a break & i don't know what i was writing after "say that" So yeah.

Oh, I remember,

Here you can say that "b" is the base price of the house & w_1, w_2, w_3 are variable that it depends on, for example, let's say w_2 is no of bedrooms so if w_2 is higher then the price of the house will be higher too.

if you have "n" features, then the model will look like,

$$f_{w,b}(X) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

$$\vec{w} = [w_1, w_2, \dots, w_n] \text{ \{vector } w\}}$$

$$\vec{x} = [x_1, x_2, x_3, \dots, x_n]$$

now, you can write

$$f_{w,b}(X) = \vec{w} \cdot (\vec{x})^T + b$$

Vectorization

⇒ Vectorization will make your code shorter & efficient.

⇒ Vectorization is used because it is much more faster on GPU's & you better learn this shit

Without vectorization,

$$\text{Prediction} = f_{\vec{w}, b}(\vec{x}) = \sum_{j=1}^n w_j x_j + b, \text{ let's say } n = \text{no of features}$$

normal code ⇒

$$f = 0$$

for j in range(n):

$$f = f + w[j] * x[j]$$

$$f = f + b$$

Vectorization,

$$f_{\vec{w}, b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

also, from in code ⇒

$$\{ F = \text{np.dot}(w, x) + b \}$$

This is fast because numpy uses parallel processing.

ex⇒ without vectorization

$$w_1 = w_1 - 0.1 d_1$$

$$w_2 = w_2 - 0.1 d_2$$

⋮

$$w_{16} = w_{16} - 0.1 d_{16}$$

With vectorization

$$\vec{w} = \vec{w} - 0.1 \vec{d}$$

Feature Scaling

⇒ when you have a large feature value & a small feature value then you might have trouble finding the minima & it might take a longer time to actually find it

let's say , $\hat{y} = w_1x_1 + w_2x_2 + b$

$$x_1 = 10000$$

$$x_2 = 10$$

Here you perform feature scaling.

A simple feature scaling is done by :-

ex ⇒

$$300 \leq x_1 \leq 2000$$

$$0 \leq x_2 \leq 5$$

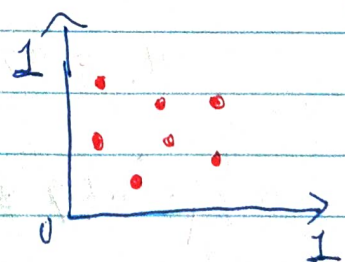
$$x_{1, \text{scaled}} = \frac{x_1}{2000}$$

$$x_{2, \text{scaled}} = \frac{x_2}{5}$$

$$0.15 \leq x_{1, \text{scaled}} \leq 1$$

$$0 \leq x_{2, \text{scaled}} \leq 1$$

⇒ This is called dividing by maximum scaling, where you divide each value of training set with the maximum value. it will look like,



[Mean normalization]

⇒ in the "dividing by maximum scaling", you only get +ve values " >0 ", but in mean normalization, you will get values " <0 " -ve values too.

ex ⇒ $300 \leq x_1 \leq 2000$

$0 \leq x_2 \leq 5$

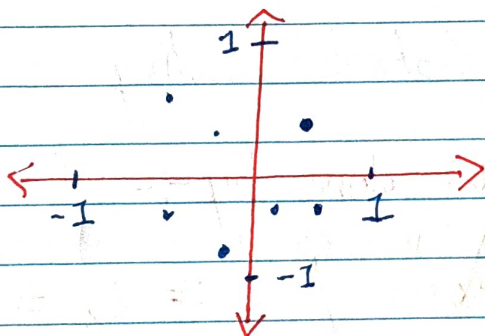
$$x_1 = \frac{x_1 - \mu_1}{2000 - 300}$$

$$-0.18 \leq x_1 \leq 0.82$$

$$x_2 = \frac{x_2 - \mu_2}{5 - 0}$$

$$-0.46 \leq x_2 \leq 0.54$$

in general, $x_i = \frac{x_i - \mu}{x_{\max} - x_{\min}}$



[Z-Score normalization]

"Z-score is not sensitive to outliers but mean normalization is because, mean normalization uses range which depends on outliers"

ex ⇒ $300 \leq x_1 \leq 2000$

$0 \leq x_2 \leq 5$

$$x_1 = \frac{x_1 - \mu_1}{\sigma_1}$$

$$x_2 = \frac{x_2 - \mu_2}{\sigma_2}$$

⇒ $-0.67 \leq x_1 \leq 3.1$

$-1.6 \leq x_2 \leq 1.9$

When you are rescaling,

aim for about, "each

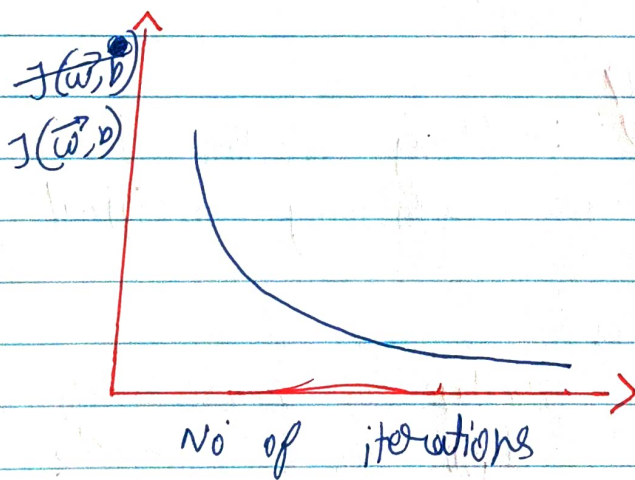
$$-1 \leq x_j \leq 1 \text{ for feature } x_j$$

but if you have ranges like

$$\left. \begin{array}{l} -3 \leq x_j \leq 3 \\ -0.3 \leq x_j \leq 0.3 \end{array} \right\} \text{acceptable ranges.}$$

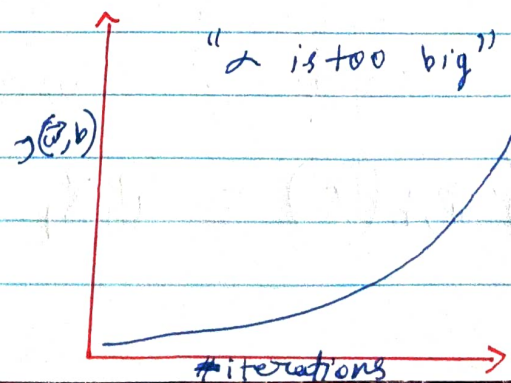
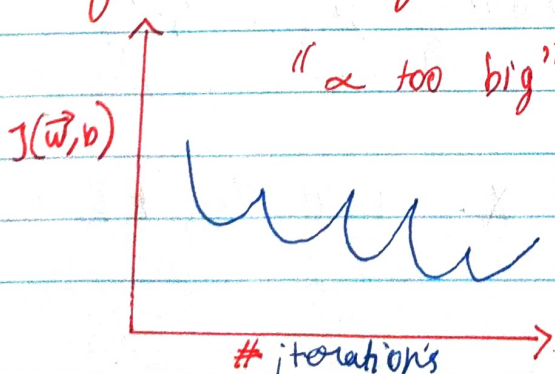
Checking gradient descent for convergence.

⇒ You can do this by using learning curve,



This curve on the graph is called learning curve, that tells you cost after every iteration when you feel the line is steepest you should stop.

Choosing the learning rate



So,

for a better learning rate $\therefore$
you should try values of α from

0.001

$\hookrightarrow \times 3$

0.01

$\hookrightarrow \times 3$

0.1

$\hookrightarrow \times 3$

1

$\hookrightarrow \times 3$

What you can do is multiply the current learning rate by 3 that will make the learning rate 3 times bigger & then try the other learning rate, because the other learning rate will be 3 times the learning rate of the current learning rates multiplied by 3.

This means you are choosing α 3 times bigger everytime.

Feature engineering

$\therefore$ Using intuition to design new features, by transforming or combining original features.

ex $\Rightarrow$ let's say,

frontage of house = x_1
depth " " = x_2

$$f(\vec{w}, b)(\vec{x}) = w_1 x_1 + w_2 x_2 + b$$

area = frontage $\times$ depth,

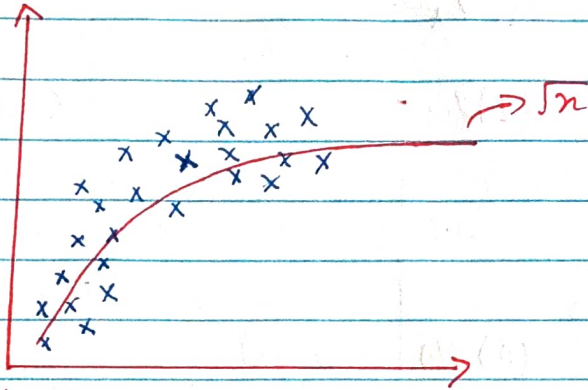
now, $x_3 = x_1 x_2$

area of house = x_3 { new feature }

$$f(\vec{w}, b)(\vec{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

Polynomial Regression-:

In real world problems data might not be linear, it curves in such cases you need Polynomial Regression.



Choice of features-:

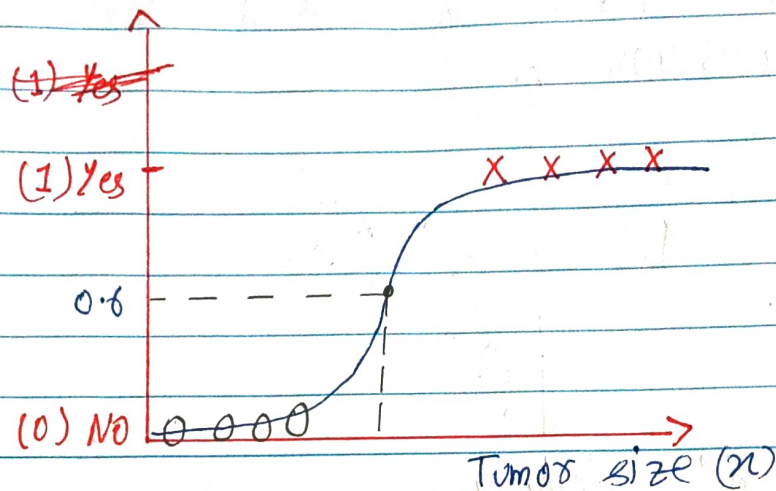
$\Rightarrow$ You can choose features to get the curve that best fits the model.

ex $\Rightarrow$

$$f_{\vec{w}, b}(x) = w_1 x + w_2 \sqrt{x} + b$$

$\Rightarrow$ So what features ($x^2, x^3, \sqrt{x}$) to use so that our model best fits the data? in course 2

Logistic Regression for Classification



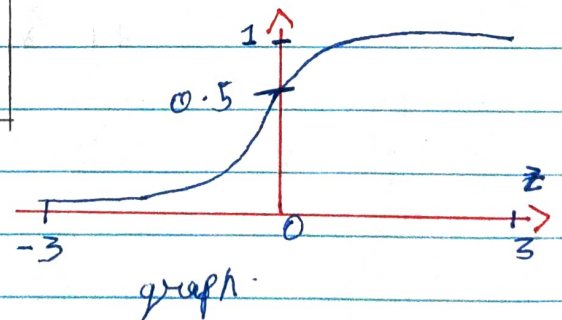
⇒ it gives you the output between zero or "1"

⇒ Say you get the output "0.6" it means that there is 60% chance that the tumor is malignant.

⇒ it works like linear regression just sigmoid function slapped on top of it, meaning if you get an output from $y = mx + b$ just apply " σ " function on it & you will get the values between zero or one.

$$\sigma = \frac{1}{1 + e^{-z}}$$

$$0 < \sigma < 1$$



So, logistic regression is nothing but the output of linear regression put in the sigmoid function.

So,

$$f_{\vec{w}, b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

→ This is logistic regression.

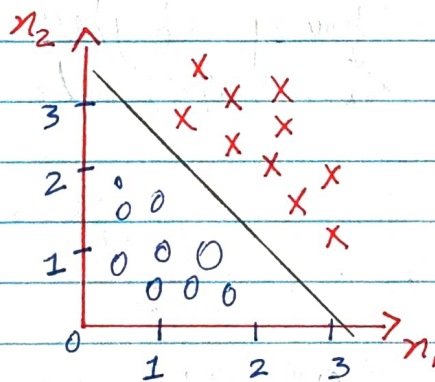
Sometimes in classification you will get to see the notation,

$$f_{\vec{w}, b}(\vec{x}) = P(y=1 | \vec{x}; \vec{w}, b)$$

it means,

Probability of y being 1 given the input variable " x " & weight & biases.

Decision Boundary



lets say,

$$f_{\vec{w}, b}(\vec{x}) = g(z) \\ = g(w_1 x_1 + w_2 x_2 + b)$$

when,

$$z = \vec{w} \cdot \vec{x} + b = 0,$$

this line is called decision boundary.

ex → lets ~~not~~ $w_1 = 1, w_2 = 2, b = -3$
then,

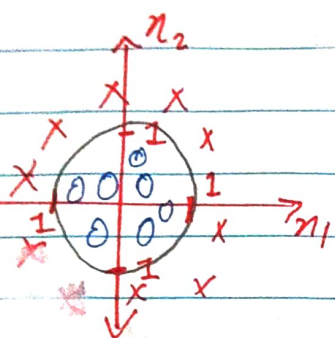
$$w_1 x_1 + w_2 x_2 + b = 0$$

$$1 \cdot x_1 + 1 \cdot x_2 + (-3) = 0$$

$$\boxed{x_1 + x_2 = 3}$$

→ This line separate two features.

Non-linear decision boundaries:-



$$f_{\vec{w}, b}(\vec{x}) = g(z) = g(w_1 x_1^2 + w_2 x_2^2 + b)$$

if you have to make a decision boundary which is a ~~circle~~ non linear in this case of a circle you can do that by having different choice features like " x_1^2 & x_2^2 " with the help of feature engineering.

in this case if " $x_1^2 + x_2^2 \geq 1$ " you predict 1 or malignant & when " $x_1^2 + x_2^2 < 1$ " you predict 0 or benign.

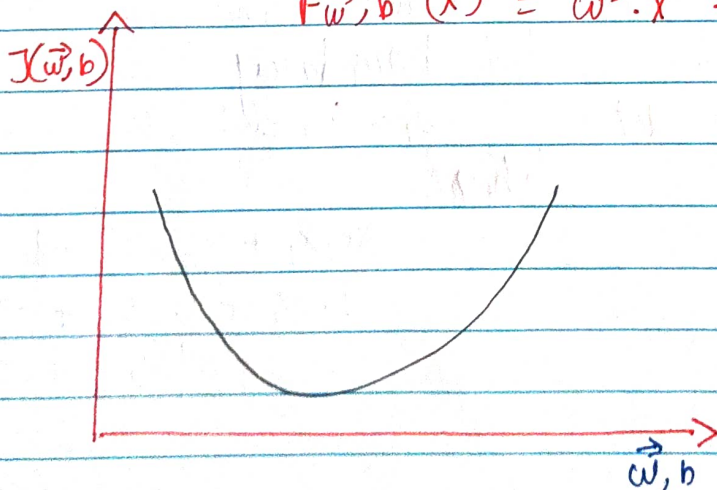
you can come up with even more complex decision boundaries by having higher order polynomial terms like " x_1^3 , x_2^4 , $x_1 x_2$ " etc

Cost function for Logistic Regression.

Here target y is "0" or "1".

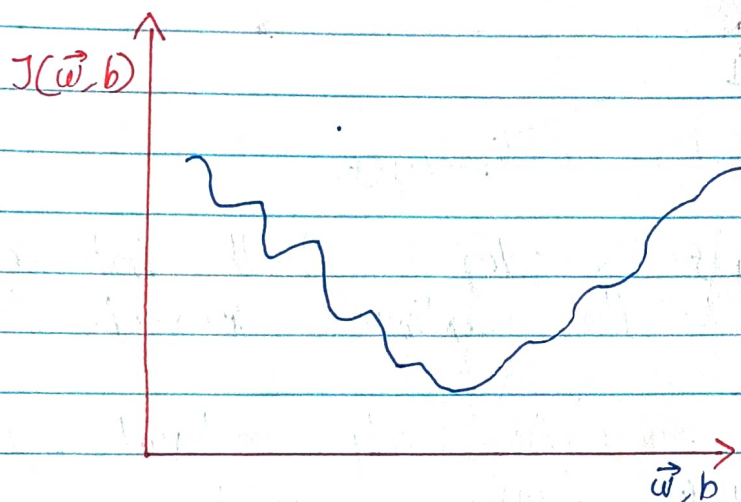
In linear regression the cost function is a convex function:

$$F_{\vec{w}, b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$



In logistic regression the cost function will look like $\therefore$ a non convex function

$$f_{\vec{w}, b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$



$\rightarrow$ Here you can get stuck at ~~global~~ local minima's.

So, we need to change the cost function for logistic regression.

Now,

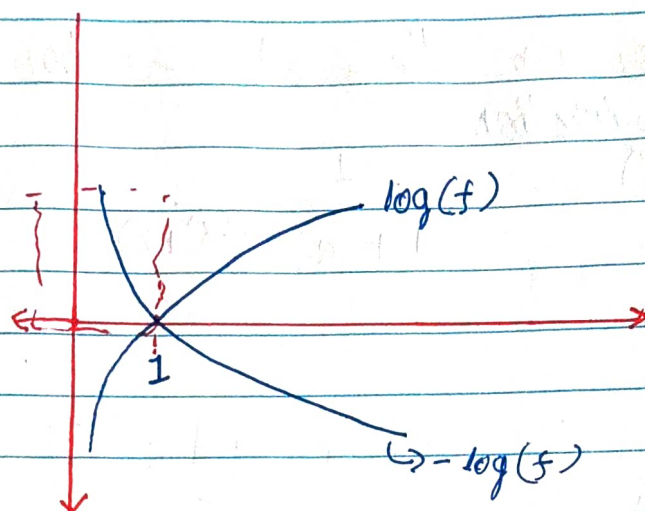
$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \left[\frac{1}{2} (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 \right]$$

for one training example let's say

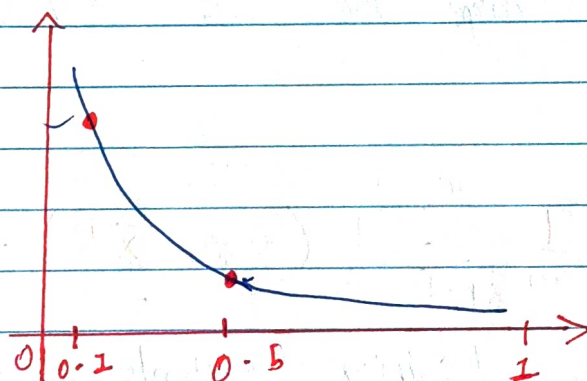
for one training set loss = $L(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)})$

Logistic loss function

$$L(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\vec{w}, b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w}, b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$



Here, the fn $\log(f)$ & $-\log(f)$ both intersect each other at one & the continues upwards & downwards respectively, so the only relevant part of the fn is $-\log(f)$ from 0 to 1,



if $y^{(i)} = 1$

& $f(\vec{w}, b(\vec{x}^{(i)})) \rightarrow 1$ (then loss $\rightarrow 0$)

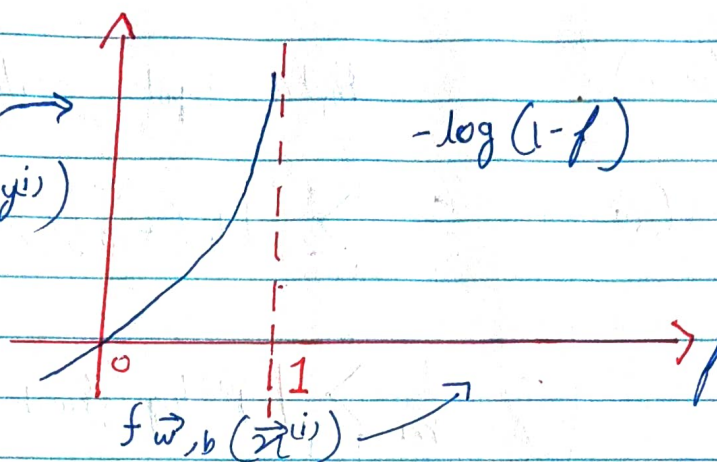
if the algorithm predicts 0.5, then the loss is a bit higher but not that high,

but if the algorithm predicted 0.1, then the loss is much higher.

if $y^{(i)} = 0$

$$L(f \vec{w}, b(\vec{x}^{(i)}), y^{(i)})$$

if $y^{(i)} = 0$



if $f(\vec{w}, b \vec{x}^{(i)}) \rightarrow 1$

then $\text{loss} \rightarrow \infty$

The further prediction $f \vec{w}, b(\vec{x}^{(i)})$ is from target $y^{(i)}$, the higher the loss.

Simplified cost function for logistic regression.

So we know,

$$L(f \vec{w}, b(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f \vec{w}, b(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f \vec{w}, b(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

$$L(f \vec{w}, b(\vec{x}^{(i)}), y^{(i)}) = -y^{(i)} \log(f \vec{w}, b(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f \vec{w}, b(\vec{x}^{(i)}))$$

if $y^{(i)} = 1$: then

$$-1 \log(f \vec{w}, b(\vec{x}^{(i)})) \stackrel{0 \leq (1-1)}{=} 0$$

if $y^{(i)} = 0$: then

$$0 \leq 0 \log(f \vec{w}, b(\vec{x}^{(i)})) - (1-0) \log(1 - f \vec{w}, b(\vec{x}^{(i)}))$$

Cost, $J(\vec{w}, b)$ for multiple input features is

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m [L(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)})]$$

$$= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)}))]$$

→ final cost function for logistic regression,
& this is convex.

Gradient descent implementation in logistic regression.

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) \}$$

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

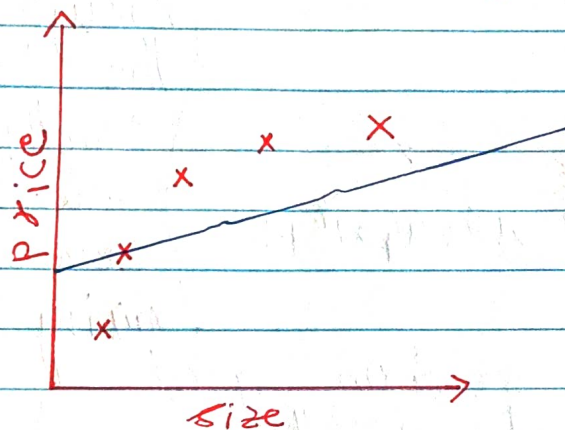
in logistic regression,

$$f_{\vec{w}, b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

Regularization in machine learning

⇒ How to counter overfitting, & underfitting

⇒ let's say we use a linear regression model in a place we were supposed to use logistic regression

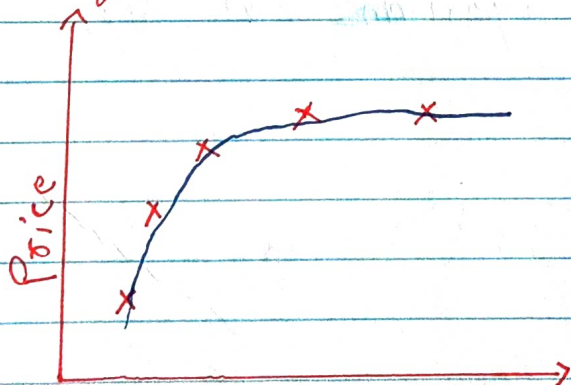


This doesn't fit the training data set well

This has "high bias" meaning it underfits the data

Size
 $w_1x + b$

Now, let's try from linear to quadratic function from fitting the data

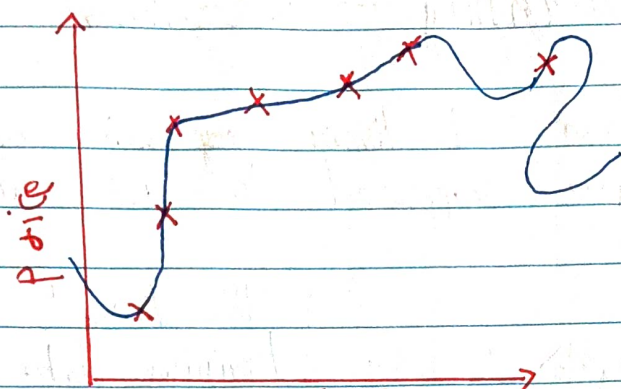


fits training set pretty well

This is called generalization meaning you want your learning algorithm to generalize well the training set & can give you a good prediction

Size $\{w_1x + w_2x^2 + b\}$

Now let's try a more complex function that can fit this data.



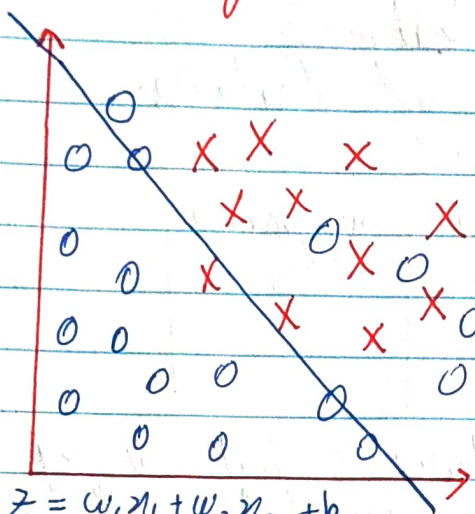
$$w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b$$

Now, with this fourth order polynomial you can actually fit the curve that passes through all five of the training examples exactly. Here the errors will be zero on all 5 training examples.

⇒ This model would perform good on the training data & very bad on the new data - & this is ~~called~~ "overfitting" the data, because this model fits the data too well,

in the other term we say the algorithm has "high variance".

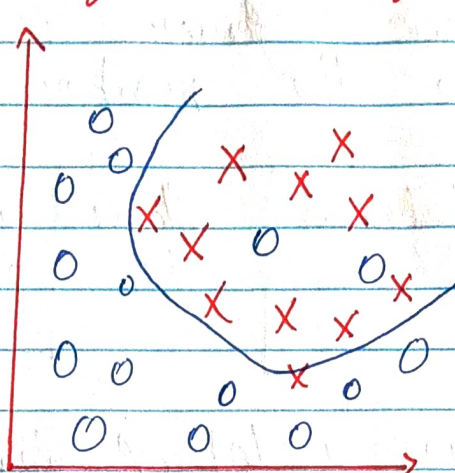
You can say the same for Classification.



$$z = w_1 x_1 + w_2 x_2 + b$$

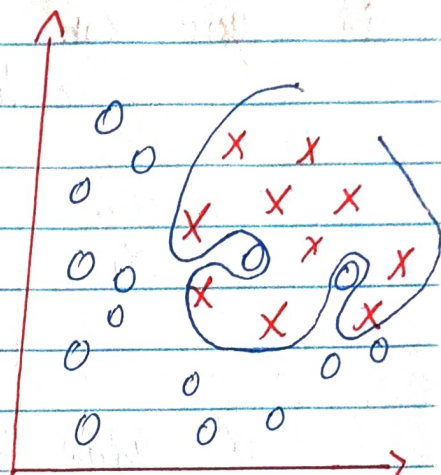
$$f(\vec{x}), b(\vec{x}) = g(z)$$

"g" is the sigmoid fn
"Underfit high bias"



$$z = w_1 x_1 + w_2 x_1^2 + w_3 x_1^2 x_2 + w_4 x_2^2 + w_5 x_1 x_2 + b$$

"This is just right"

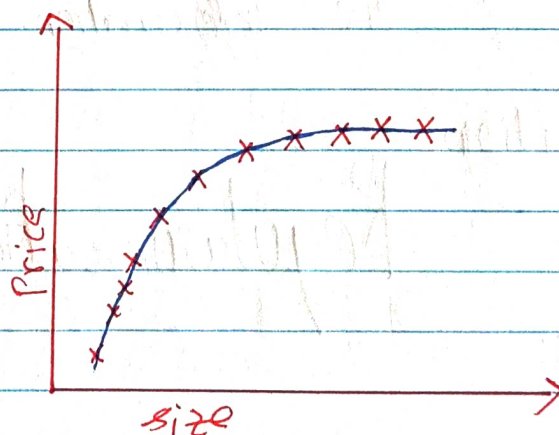
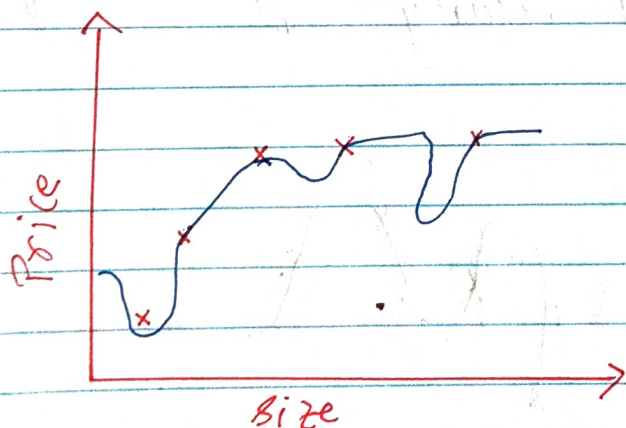


$$z = w_1 x_1 + w_2 x_2 + w_3 x_1^2 x_2 + w_4 x_1^2 x_2^2 + \dots + b$$

"Overfits the data"

Regularization to Reduce Overfitting.

① Collected more training examples



Overfit

⇒ By having more training examples you can have a function which is less wiggly.

A second option for addressing overfitting is to see if you can use fewer features.
 $\Rightarrow$ reduce the number of features.

$$\Rightarrow$$

size x_1	bedrooms x_2	floors x_3	age x_4	avg income x_5	-----	Price y
---------------	-------------------	-----------------	--------------	---------------------	-------	--------------

if you have a lot of features but very little or insufficient data, your model will overfit

So select fewer features $\therefore$

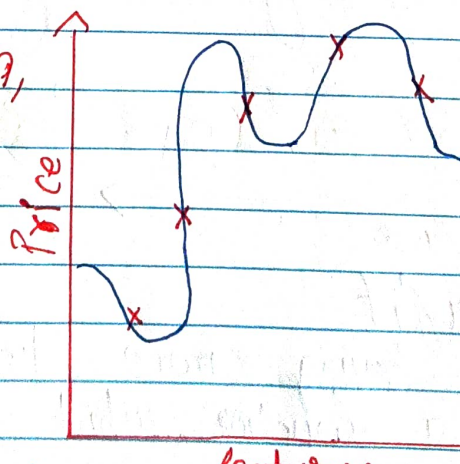
- ex: ① size
 ② Bedrooms
 ③ age

This will reduce the overfitting
 choosing the most appropriate feature is called feature selection.

The disadvantage of this algorithm is throwing not getting some of the information that it actually needs

③ option,

Regularization,



$$F(x) = 28x - 385x^2 + 39x^3 - 174x^4 + 100$$

⇒ In the model that is overfitting you find that the parameters are often relatively large,

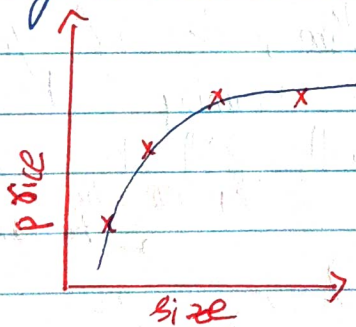
eliminate some of the features to reduce the overfitting which is setting the parameter for some feature as "0". So you can

⇒ Regularization is a way to reduce the impact of some of the ~~the~~ features more gently without setting them out right zero.

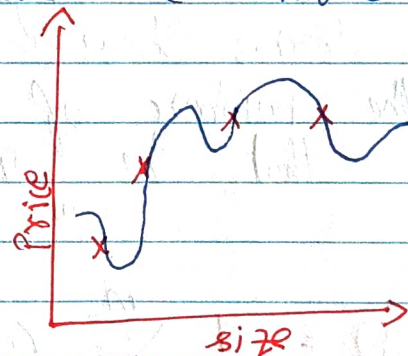
⇒ It reduces the values of the parameters. but it lets you keep ~~the values~~ all the features.

Cost function with Regularization :-

in creating a cost function with Regularization we need to make the unnecessary parameters very small so they don't make the model overfit.



$$w_1x + w_2x^2 + b$$



$$w_1x + w_2x^2 + \cancel{w_3x^3} + \cancel{w_4x^4} + b$$

$\approx 0 \approx 0$

make w_3, w_4 very small (≈ 0)

$$\Rightarrow \min_{\vec{w}, b} \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + 1000 w_3^2 + 1000 w_4^2$$

if we penalize the model for having high Parameters for

w_3 & w_4 , then the model automatically reduces the size of the parameter w_3 & w_4

Regularization:

if you have small values of $w_1, w_2, \dots, w_n$ & b , then you will end up with simpler model which is less likely to overfit.

If you have a lot of features you may not know the most important features & which ones to penalize so in Regularization we penalize all of the features's Parameters.

So, the new cost function will be,

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

here λ "lambda" is the regularization parameter.
 $\lambda > 0$

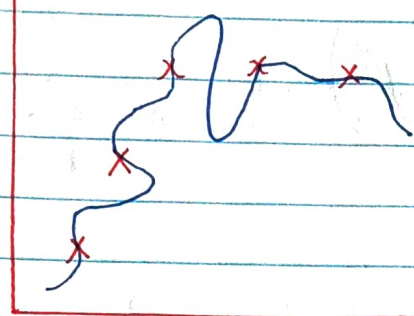
$\Rightarrow$ we divide " λ " by $2m$, so that both the first & second terms here are scaled by $\frac{1}{2m}$, & if you increase the number of testing data the training set's it is likely that the value of " λ " will still be effective.

So,

$$J(\vec{w}, b) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2}_{\text{fit's data}} + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^n w_j^2}_{\text{regularization term}}$$

$\Rightarrow$ " λ balances both bat games"

if you pick, $\lambda = 0$ { extreme value }



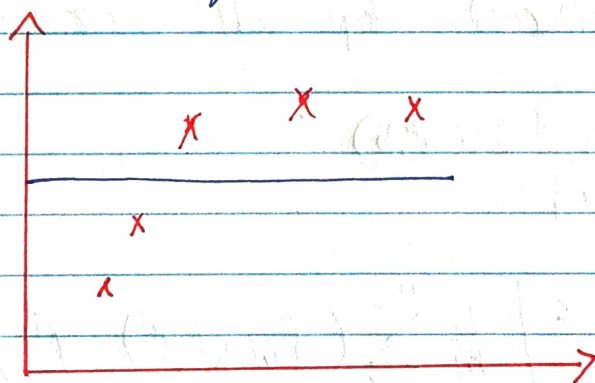
if $\lambda = 0$, then there will be no penalizing of the parameters & hence the model overfits the data.

$$f_{\vec{w}, b}(\vec{x}) = w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b$$

if you pick, very large lambda " λ ", let's say, $\lambda = 10^{10}$

$$f_{\vec{w}, b}(\vec{x}) = \overset{\rightarrow 0}{w_1x} + \overset{\rightarrow 0}{w_2x^2} + \overset{\rightarrow 0}{w_3x^3} + \overset{\rightarrow 0}{w_4x^4} + b$$

then, the model chooses the values of w_1, w_2, w_3, w_4 very very small & we are left with " b " the bias which results in a straight line. & it underfits the data.



Regularized Linear Regression

$$\min_{(\vec{w}, b)} J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m \left(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

Now, derivating w.r.t "w" we get.

$$\frac{\partial J(\vec{w}, b)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m \left(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{m} w_j$$

for "b" we will use the simple cost function we don't need the Regularized cost function.

$$\frac{\partial J(\vec{w}, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m \left(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} \right)$$

Now, the gradient descent step will look like.

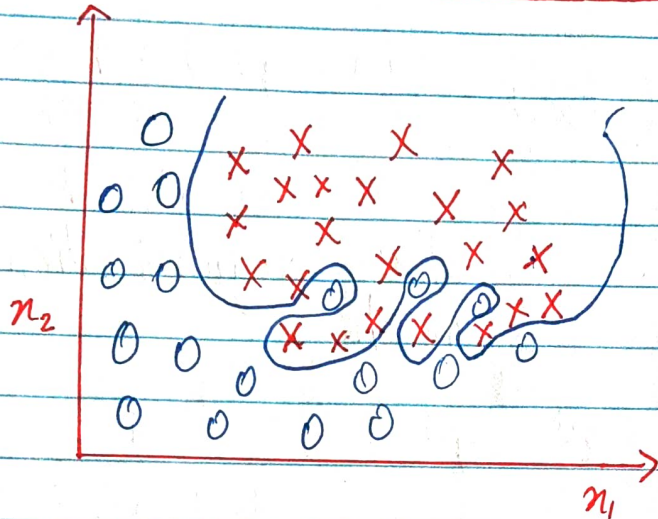
$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m \left(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{m} w_j \right]$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m \left(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} \right)$$

Regularized Logistic Regression



$$z = w_1 x_1 + w_2 x_2 + w_3 x_1^2 x_2 + w_4 x_1^2 x_2^2 + w_5 x_1^2 x_2^3 + \dots + b$$

$$f_{\vec{w}, b}(\vec{x}) = \frac{1}{1 + e^{-z}}$$

⇒ Here, z is a very complex polynomial & you can end up with a decision boundary that is overly complex.

So, to reduce that here is the regularized cost function.

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)})) \right] + \frac{1}{2m} \sum_{j=1}^n w_j^2$$

we will use this cost function to prevent the ^{model} from overfitting.

Gradient descent for regularized logistic regression.

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \quad \left| \quad \frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right.$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

$$\left| \quad \frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) \right.$$

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \left[\frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right] + \left[\frac{\lambda}{m} w_j \right]$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) \quad \left. \vphantom{\frac{\partial}{\partial b} J(\vec{w}, b)} \right\} \begin{array}{l} \text{bias will} \\ \text{not change} \\ \text{as it doesn't affect} \\ \text{the regularization as} \\ \text{much.} \end{array}$$

Here, $f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)} =$ ~~$\frac{1}{1 + e^{-(\vec{x}^{(i)} - y^{(i)})}}$~~

~~$$\frac{1}{1 + e^{-(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})}}$$~~

$$\frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$